you are given a held-out test set (which has not been seen before by the network), on which you have to report the predicted output. What is the drop-out rate that you would employ on the inner layers for the test set predictions? The answer, of course, is that one does not employ any drop-out on the test set.

Many of the interviewees fumble at this question. The key point to remember is that drop-out is employed basically to enable the network to generalise better by preventing the over-dependence on any particular set of units being active, during training. During test set prediction, we do not want to miss out on any of the features getting dropped out (which would happen if we use drop-out and prevent the corresponding neural network units from activating on the test data signal), and hence we do not use drop-out. An additional question that typically gets asked is: What is the inverted drop-out technique? I will leave it for our readers to find out the answer to that question.

Another question that frequently gets asked is on splitting the data set into train, validation and test sets. Most of you would be familiar with the nomenclature of train, validation and test data sets. So I am not going to explain that here. In classical machine learning, where we use classifiers such as SVMs, decision trees or random forests, when we split the available data set into train, validation and test, we typically use a split, 60-70 per cent training, 10-20 per cent validation and 10 per cent test data. While these percentages can vary by a few percentage points, the idea is to have to validate and test data sizes that are 10-20 per cent of the overall data set size. In classical machine learning, the data set sizes are typically of the order of thousands, and hence these sizes make sense.

Now consider a deep learning problem for which we have huge data sets of hundreds of thousands. What should be the approximate split of such data sets for training, validation and testing? In the Big Data sets used in supervised deep learning networks, the validation and test data sets are set to be in the order of 1-4 per cent of the total data set size, typically (not in tens of percentage as in the classical machine learning world).

Another question could be to justify why such a split makes sense in the deep learning world, and this typically leads to a discussion on hyper-parameter learning for neural networks. Given that there are quite a few hyper-parameters in training deep neural networks, another typical question would be the order in which you would tune for the different hyper-parameters. For example, let us consider three different hyper-parameters such as the mini-batch size, choice of activation function and learning rate. Since these three hyper-parameters are quite inter-related, how would you go about tuning them during training?

We have discussed quite a few machine learning questions till now; so let us turn to text analytics.

Given a simple sentence 'S' such as, "The dog chased the young girl in the park," what are the different types of text analyses that can be applied on this sentence in an increasing order of complexity? The first and foremost thing to do is basic lexical analysis of the sentence, whereby you identify the lexemes (the basic lexical analysis unit) and their associated part of the speech tags. For instance, you would tag 'dog' as a noun, 'park' as a noun, and 'chase' as a verb. Then you can do syntactic analysis, by which you combine words into associated phrases and create a parse tree for the sentence. For instance, 'the dog' becomes a noun phrase where 'the' is a determiner and 'dog' is a noun. Both lexical and syntactic analysis is done at the linguistic level, without the requirement for any knowledge of the external world.

Next, to understand the meaning of the sentence (semantic analysis), we need to identify the entities and relations in the text. In this simple sentence, we have three entities, namely 'dog', 'girl', and 'park'. After identifying the entities, we also identify the classes to which they belong. For example, 'girl' belongs to the 'Person' class, 'dog' belongs to the 'Animal' class and 'park' belongs to the 'Location' class. The relation 'chase' exists between the entities 'dog' and 'girl'. Knowing the entity classes allows us to postulate the relationship between the classes of the entities. In this case, it is possible for us to infer that the 'Animal' class entity can 'chase' the 'Person' class entity. However, semantic analysis involving determining entities and the relations between them, as well as inferring new relations, is very complex and requires deep NLP. This is in contrast to lexical and syntactic analysis which can be done with shallow NLP.

Deep NLP requires common sense and a knowledge of the world as well. The major open challenge in text processing with deep NLP is how best we can represent world knowledge, so that the context can be appropriately inferred. Let us consider the sentence, "India lost for the first time in a cricket test match to Bangladesh." Apart from the literal meaning of the sentence, it can be inferred that India has played with Bangladesh before, that India has beaten Bangladesh in previous matches, etc. While such inferences are very easy for humans due to our contextual or world knowledge, machines cannot draw these inferences easily as they lack contextual knowledge. Hence, any efficient NLP system requires representation of world knowledge. We will discuss this topic in greater detail in next month's column.

If you have any favourite programming questions/ software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Till we meet again next month, wishing all our readers a wonderful and productive year! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist at Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at *http://www.linkedin.com/groups?home=&gid=2339182*